Fourier Series & Continuous Fourier Transform

1 Task1: Redrawing a 2D shape with Fourier Series

1.1 Overview

In this offline assignment, you will use the **complex (exponential) form of the Fourier Series** to redraw a 2D shape as a sum of vectors rotating at constant speed – the classic "Fourier epicycle" animation.

Everything that is *not* the Fourier Series itself – reading an input SVG drawing, and drawing/animating the rotating circles – is provided for you. Your job is only the four methods described in Subsection 1.3.1.

1.2 Background

A 2D drawing traced out over one period can be written as a single complex-valued periodic function of time,

$$f(t) = x(t) + j\,y(t), \qquad 0 \le t \le T,$$

where $(x(t), y(t))$ is the pen position at time t and T is the period. Just as in the real (sine/cosine) form of the Fourier Series, $f(t)$ can be written as an infinite sum – but now of *rotating complex exponentials* instead of sines and cosines:

$$f(t) = \sum_{n=-\infty}^{\infty} c_n\, e^{jn\omega t}, \qquad \omega = \frac{2\pi}{T}. \tag{1}$$

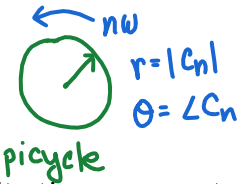
The complex Fourier coefficients c_n are computed by

$$c_n = \frac{1}{T}\int_0^T f(t)\, e^{-jn\omega t}\, dt. \tag{2}$$

Note that n ranges over *all* integers, positive, negative, and zero – unlike the real form, negative harmonics are not redundant here and must not be dropped.

Why "epicycles"? Each term $c_n e^{jn\omega t}$ in Eq. (1) is a vector of fixed length $|c_n|$ and fixed initial angle $\arg(c_n)$, rotating at constant angular speed $n\omega$. If you place these vectors tip-to-tail (in any order), the free end traces out the N-term partial sum

$$\hat{f}_N(t) = \sum_{n=-N}^{N} c_n\, e^{jn\omega t}$$



at every instant t. Each rotating vector, together with the circle its tip sweeps out, is called an **epicycle**. Figure 1 shows a snapshot of this construction partway through drawing a heart: the

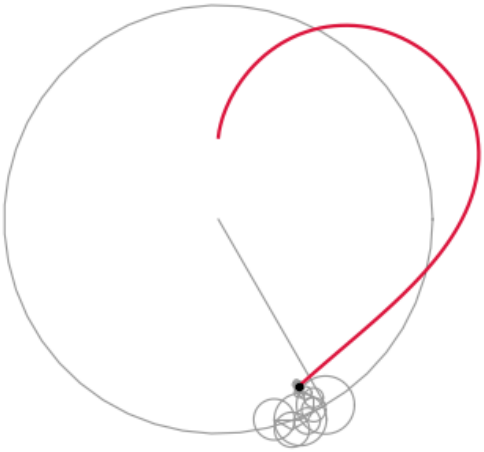


Figure 1: A snapshot of the epicycle animation partway through tracing a heart shape. The largest circle corresponds to the dominant harmonic; each subsequent, smaller circle is centered on the tip of the previous one.

chain of grey circles is the sum of rotating vectors, the black dot is the current tip position $\hat{f}_N(t)$, and the red curve is the path traced by that tip so far.

Figure 2 shows what you are ultimately checking your implementation against: the original traced shape versus the reconstruction $\hat{f}_N(t)$ obtained by summing all $2N + 1$ terms.

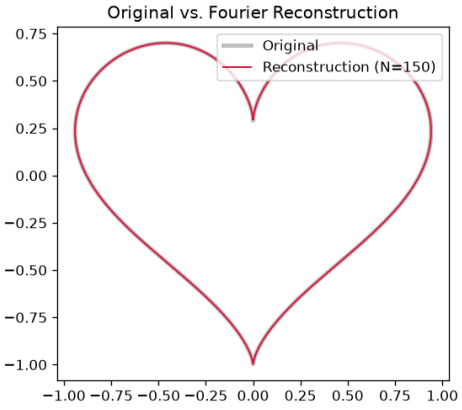


Figure 2: Original drawing (grey) versus the Fourier reconstruction (red) using $N = 150$ harmonics.

1.3 Problem Description

You are tasked with implementing a Python class, `FourierEpicycles`, that computes the complex Fourier coefficients of a sampled periodic signal and reconstructs an approximation of it. The provided utility code turns this reconstruction into the rotating-circle animation and saves it as a GIF.

1.3.1 Requirements

Implement the following methods of `FourierEpicycles` in `fs_redrawer.py`:

1. `__init__(self, t, signal, n_harmonics)`: store the sampled time array `t`, the complex-valued sampled signal `signal`, and the number of harmonics N (`n_harmonics`). Also compute and store the period T and the fundamental angular frequency $\omega = 2\pi/T$, and initialize an empty coefficient store.

2. `calculate_cn(self, n)`: compute the single complex coefficient c_n from Eq. (2) using numerical integration.

3. `calculate_all_coefficients(self)`: compute and store c_n for every harmonic $n = -N, \ldots, N$.

4. `approximate(self, t)`: evaluate the partial sum $\hat{f}_N(t) = \sum_{n=-N}^{N} c_n e^{jn\omega t}$ at the given time(s) `t`, where `t` is a NumPy array (it should also work for a single scalar `t`, since your formula is the same either way).

No other method needs to be written or modified.

1.3.2 Provided Utilities

Two modules are provided alongside the template and are already imported by it. You do not need to read, understand, or modify their internals – treat them as a black box with the interface described below.

`svg_utils.py`

```
t, z = load_svg_path(filename, num_points=1000)
```

Loads a single-path SVG drawing and returns:

- `t`: a NumPy array of `num_points` time samples, uniformly spaced over the *closed* interval $[0, 2\pi]$ (so `t[0] == 0` and `t[-1] == 2*np.pi`).
- `z`: a complex NumPy array of the same length, with `z[i] == x(t[i]) + 1j*y(t[i])`, centered at the origin and scaled to roughly fit in $[-1, 1] \times [-1, 1]$. It is periodic: `z[-1] == z[0]`.

Pass `t` and `z` straight into your `FourierEpicycles` constructor.

`epicycle_animation.py`

```
save_outputs(fs, z_true, comparison_path, gif_path, num_frames=240, fps=30)
```

Takes your (already-computed) `fs` object – it reads `fs.coeffs`, `fs.omega`, `fs.T`, and calls `fs.approximate(...)` – and saves two files: a static overlay PNG to `comparison_path` (Figure 2) and the rotating-circle animation as a GIF to `gif_path` (Figure 1). The provided `__main__` block in `fs_redrawer.py` already calls this for you – there is nothing to add here.

1.3.3 Input Drawings

Four sample drawings are provided in the `svgs/` folder: `star.svg`, `heart.svg`, `infinity.svg`, and `circle.svg`. Your implementation must work correctly on all of them (and, in principle, on any similar single-path SVG drawing) without any shape-specific code – everything shape-specific is already handled inside `load_svg_path`. The expected output for each of these four drawings (both the comparison PNG and the epicycle GIF, produced by a correct implementation with $N = 150$) is provided in the `expected_outputs/` folder alongside this specification, so you can check your own output against it.

Run your completed file from the command line, passing the path to an SVG file and, optionally, the number of harmonics (default 150) and explicit output paths for the comparison PNG and the GIF (default to `<name>_comparison.png` and `<name>_epicycles.gif` in the current directory):

```
python3 fs_redrawer.py svgs/heart.svg 150 heart_comparison.png heart_epicycles.gif
```

Open the GIF in any image viewer to watch the epicycles draw the shape.

1.4 Implementation Guidelines

- Use numerical integration for Eq. (2). Use `np.trapezoid` (this is the modern replacement for `np.trapz`, which has been removed in recent NumPy versions – if your installed NumPy still has `np.trapz`, it is equivalent and also acceptable).
- Because `t` spans the *closed* interval $[0, T]$ with `signal[-1] == signal[0]`, a plain call to `np.trapezoid(signal * kernel, t)` already integrates correctly over one full period – no extra "wrap-around" handling is needed.
- You may not use `numpy.fft` (or any other Fast Fourier Transform routine), `numpy.correlate`, or any other built-in routine that performs the harmonic decomposition for you. c_n must be computed by numerically integrating the sampled signal against $e^{-jn\omega t}$ yourself.
- Do not hard-code closed-form/analytical Fourier coefficients for any of the provided shapes. Your `calculate_cn` must work for an arbitrary sampled signal, not just the four shapes provided.
- `approximate(t)` must be vectorized: the static comparison plot calls it with an array of thousands of times at once, so it must not assume `t` is a single number.
- Do not forget the negative harmonics ($n < 0$) in `calculate_all_coefficients` – omitting them will visibly distort every reconstruction.

2 Task2: Edge detection with Continuous Fourier Transform

2.1 Overview

In this offline assignment, you will use the **Two-Dimensional Continuous Fourier Transform (2D-CFT)** to perform *frequency-domain edge detection* on a grayscale image. Edges and fine texture in an image correspond to abrupt, fast-changing intensity variation – that is, *high spatial frequency* content. Smooth regions (sky, skin, fur, background) vary slowly and correspond to *low spatial frequency* content clustered near the origin of the frequency plane. If you transform an

image into the frequency domain, suppress everything within some radius of the origin (a **high-pass filter**), and transform back, what remains is, to a good approximation, an edge map of the original image.

Everything that is *not* the transform itself – loading and normalizing the input image, and applying the high-pass filter – is provided for you. Your job is only the three methods described in Subsection 2.3.2.

2.2 Background

Let $I(x, y)$ represent a continuous grayscale image defined over spatial coordinates (x, y). The 2D Continuous Fourier Transform of $I(x, y)$ is defined as

$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} I(x, y)\, e^{-j2\pi(ux+vy)}\, dx\, dy,$$

where (u, v) are the spatial frequencies conjugate to (x, y). Using Euler's identity, this splits into a real and an imaginary part that can each be evaluated with ordinary (real-valued) numerical integration:

$$\Re\{F(u, v)\} = \iint I(x, y)\, \cos\big(2\pi(ux + vy)\big)\, dx\, dy, \tag{3}$$

$$\Im\{F(u, v)\} = -\iint I(x, y)\, \sin\big(2\pi(ux + vy)\big)\, dx\, dy. \tag{4}$$

The inverse 2D-CFT reconstructs the spatial-domain image from its frequency-domain representation:

$$I(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} F(u, v)\, e^{j2\pi(ux+vy)}\, du\, dv. \tag{5}$$

Separability. A direct evaluation of Eq. (3)–(4) on an $N \times N$ pixel grid, for every (u, v) pair, is an $O(N^4)$ computation and is not practical. Instead, expand $\cos(2\pi(ux+vy))$ and $\sin(2\pi(ux+vy))$ with the angle-sum identities and observe that the double integral separates into two *sequential 1D integrals*: first integrate over x for every row and every candidate u, then integrate the result over y for every candidate v (and symmetrically for the inverse transform). Each stage is $O(N^3)$, which is what makes the computation tractable using only `np.trapezoid`.

Why does a high-pass filter isolate edges? The magnitude spectrum $|F(u, v)|$ of a typical image is heavily concentrated near the origin $(u, v) = (0, 0)$ – see Figure 3 – because most of an image's energy is in slowly-varying or constant brightness (flat regions, backgrounds), not sharp detail. Zeroing out $F(u, v)$ for $\sqrt{u^2 + v^2}$ below some cutoff radius removes exactly that slowly-varying content while leaving the high-frequency components – outlines, edges, texture – intact. Inverse-transforming what remains reconstructs (an approximation of) just that high-frequency content.

2.3 Problem Description

You are required to perform the following steps:

1. Load and normalize a grayscale image, and build its continuous spatial *and* frequency coordinate axes. (Given)

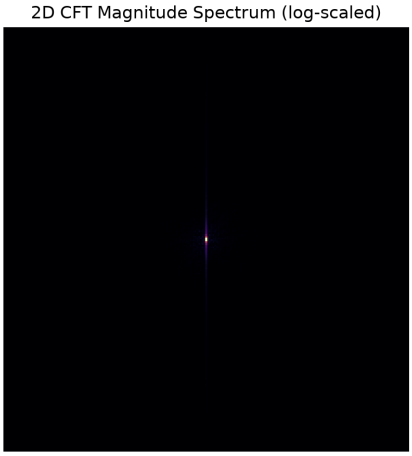


Figure 3: Log-scaled magnitude spectrum $\log(1 + |F(u, v)|)$ of the provided sample image. Note how sharply the energy is concentrated near the center – a high-pass filter only needs to zero out a small central disk to remove almost all of the low-frequency (non-edge) content.

2. Compute the 2D Continuous Fourier Transform using separable numerical integration.
3. Visualize the magnitude spectrum of the transformed image.
4. Apply a frequency-domain high-pass filter. (Given)
5. Reconstruct the filtered result using the inverse 2D-CFT, producing the edge map.

2.3.1 Object-Oriented Design Specifications

Your implementation must follow the class structure below, provided in `cft_edge_detector.py`.

Class: `ContinuousImage` **(Given)** Represents a grayscale image as a continuous spatial signal.

- **Attributes:** `image` (normalized 2D intensity array), `x`, `y` (continuous spatial coordinate vectors).
- **Methods:** `show(title)` – displays the image.

Class: `CFT2D` Computes the 2D Continuous Fourier Transform of a given image.

- **Attributes:** `I` (image intensity matrix), `x`, `y` (spatial coordinate vectors), `u`, `v` (frequency coordinate vectors, already constructed for you in `__init__` to span the full Nyquist range implied by the pixel spacing of `x`/`y` – *use these, not* `x`/`y`*, as the frequency axes*).
- **Methods:**
  - `compute_cft()` – computes and returns the real and imaginary components of the 2D-CFT, using separable trapezoidal integration.
  - `plot_magnitude()` – visualizes the logarithmic magnitude spectrum.

Class: `FrequencyFilter` (Given) Applies frequency-domain filtering operations.

- **Methods:** `high_pass(real, imag, cutoff)` – zeroes out all frequency components within radius `cutoff` (in pixel-index units, measured from the spectrum's center) of the origin, suppressing low-frequency content and preserving high-frequency (edge) content.

Class: `InverseCFT2D` Reconstructs the spatial-domain signal from a 2D frequency spectrum.

- **Attributes:** `real`, `imag` (filtered frequency-domain components), `u`, `v` (frequency coordinate vectors the spectrum was computed on), `x`, `y` (spatial coordinate vectors to reconstruct onto).
- **Methods:** `reconstruct()` – performs the inverse 2D numerical integration and returns the reconstructed (real-valued, possibly negative) signal.

2.3.2 Requirements

Implement the following methods in `cft_edge_detector.py`:

1. `CFT2D.compute_cft(self)`: computes $\Re\{F(u,v)\}$ and $\Im\{F(u,v)\}$ from Eq. (3)–(4) via separable trapezoidal integration over `self.u`, `self.v`, `self.x`, `self.y`.
2. `CFT2D.plot_magnitude(self)`: plots $\log(1+|F(u,v)|)$ using the spectrum returned by `compute_cft()`.
3. `InverseCFT2D.reconstruct(self)`: computes Eq. (5) via separable trapezoidal integration, returning the reconstructed spatial-domain array.

No other method needs to be written or modified. Detailed docstrings with exact expected shapes and formulas are included in the skeleton file for each of these three methods.

2.4 Input Image and Usage

A sample grayscale image, `pikachu.png`, is provided alongside this specification. Run your completed file from the command line, passing the input image path, the output image path, and (optionally) the high-pass cutoff radius in pixel-index units (default 15):

```
python3 cft_edge_detector.py pikachu.png pikachu_edges.png 15
```

The command-line entry point at the bottom of the skeleton (already written for you) takes care of turning the reconstructed signal into a displayable edge map: since the DC/low-frequency component that carried the average brightness has been removed by the high-pass filter, `reconstruct()` returns an array with both positive and negative values, so the entry point takes the absolute value, rescales it to $[0, 1]$, and inverts it so that edges come out dark on a light background before saving. Figure 4 shows the expected input/output pair for the provided image at cutoff $= 15$; your output does not need to match pixel-for-pixel, but should show a clearly recognizable edge map of the same subject.

2.5 Implementation Guidelines

- Use `np.trapezoid` for every integral (this is the modern replacement for `np.trapz`, which has been removed in recent NumPy versions – if your installed NumPy still has `np.trapz`, it is equivalent and also acceptable).

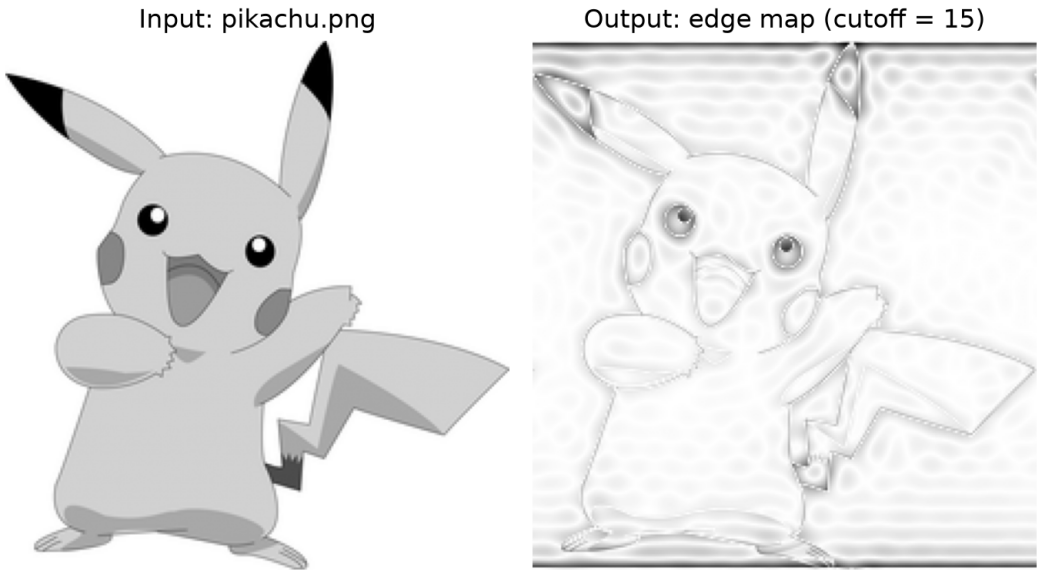


Figure 4: Expected input/output behavior of a correct implementation on the provided `pikachu.png`, with cutoff radius 15.

- You must exploit separability as described in the Background section. A direct $O(N^4)$ quadruple loop/integral over (x, y, u, v) will be far too slow and will not be accepted.
- Use of `numpy.fft`, `scipy.fft`, or any other built-in Fourier/DFT/FFT routine is strictly prohibited anywhere in this assignment.
- Complex exponentials must be handled by explicitly separating real and imaginary parts (cosine/sine), as in Eq. (3)–(4) and (5). Do not use `numpy`'s complex-number arithmetic to sidestep this.
- Do not hard-code any edge-detection formula (e.g. Sobel/Prewitt/Canny kernels, `cv2.Canny`, image gradients via finite differences, etc.). The edge map must emerge purely from your `compute_cft` → `high_pass` → `reconstruct` pipeline.
- Use `self.u`/`self.v` (not `self.x`/`self.y`) as the frequency axes in `compute_cft` – they are already constructed for you in `CFT2D.__init__` and span the correct (Nyquist) frequency range. Using `self.x`/`self.y` instead will still run, but the spectrum will be so under-resolved that no cutoff radius will produce a meaningful edge map.
- You are not allowed to initialize/declare any new classes or new attributes/methods beyond what is already defined in the skeleton. Follow the class structure given above.

3 Submission

Submit a single zip file named `<StudentID>.zip` (e.g., if your student ID is 2305041, name it `2305041.zip`). It must extract to

```
2305041/
   |- fs_redrawer.py
   |- cft_edge_detector.py
```

4 Mark Distribution

Table 1: Mark Distribution

Component	**Points**
Initialization & Logic (`__init__`)	5 points
Method `calculate_cn()`	20 points
Method `calculate_all_coefficients()`	5 points
Method `approximate()`	13 points
Correct Reconstruction Across All Provided Shapes	7 points
`CFT2D.compute_cft()` – correct separable 2D-CFT	23 points
`CFT2D.plot_magnitude()`	5 points
`InverseCFT2D.reconstruct()` – correct separable inverse 2D-CFT	17 points
Correct end-to-end edge map on the provided image	5 points
Total	**100 points**